

Claude Code inside a Node backend, routed to GLM 5.3

How to let an Express, Fastify or NestJS backend call a model through the Claude Code CLI, with Z.ai's GLM models behind it: `glm-5.3-flash[1m]` for most work, `glm-5.3[1m]` for hard work, and skills when a task needs reusable instructions. This is how the Stackup CRM (LeadWorks) does it on the server. For a new developer or for an LLM agent.

Quick answers

Where does Claude Code get installed? In the project's own `backend/node_modules`, pinned to one exact version. Never globally, never in `~/.claude`.

Where do skills go? In your repo, for example `backend/llm-skills/<name>/SKILL.md`. For each call the app copies that folder into the call's temporary `.claude/skills`.

Where does the GLM key live? In the server's `.env` only. Not in git, not in any `settings.json`.

What can the model touch? Nothing, unless you name a tool. The default is no tools at all (section 6).

Which model? Flash by default because its quota is larger. The full model for hard tasks, chosen per task kind by an env variable.

[1 What it is](#) [2 Folders](#) [3 Install](#) [4 Configure and pick models](#) [5 The runner](#)
[6 Tool access](#) [7 Express, Fastify, Nest](#) [8 Skills](#) [9 Server rules](#) [10 Traps](#) [11 Test it](#)
[12 For an LLM agent](#)

1. What you are building

Claude Code is a command-line client. It sends requests to the address in `ANTHROPIC_BASE_URL` using the key in `ANTHROPIC_API_KEY`. Z.ai offers an Anthropic-compatible address, so the same CLI talks to GLM. Your backend does not call an API library. It starts the CLI as a short-lived child process, gives it a prompt on stdin, and reads one JSON answer.

1 REQUEST

HTTP route or job

2 QUEUE

BullMQ job, because a call takes seconds to minutes

3 WORKER

Calls `runClaude()`, limited to N at a time

4 CHILD PROCESS

`claude -p` with its own tiny environment

5 Z.AI

GLM answers, JSON comes back, you validate it

Three rules hold everywhere in this design:

- **One call, one child, one temp folder.** Created before the call, deleted after it. No state, no session history survives.
- **The child never inherits your environment.** It receives a short allowlist. Your database URL and other secrets never reach a process that runs a model.
- **Never trust the output.** Validate it with a schema (zod) in your own code, whatever the CLI claims.

Not the same as using Claude Code while developing.

Typing `claude` in a terminal to help you write code is a developer tool and uses `~/.claude/skills` and `.claude/skills` of that project. This guide is about the *product* calling a model at run time on the server. The two share the binary and nothing else.

2. Where everything lives

WHAT	WHERE (SERVER AND REPO)
The project	<code>/var/www/html/<project>/</code> on the server, with <code>backend/</code> inside
Claude Code binary	<code>backend/node_modules/@anthropic-ai/claude-code</code> , from <code>package.json</code> , exact version
GLM key, URL, models	<code>backend/.env</code> on the server (<code>LLM_*</code> variables). Git-ignored.
The wrapper code	<code>backend/src/llm/claude-runner.ts</code> (section 5)
Skills (sources)	<code>backend/llm-skills/<name>/SKILL.md</code> in git (section 8)
Per-call scratch	<code>/tmp/myproject-llm-XXXX/{cwd,config,system.txt}</code> (use your own project's name as the prefix, because <code>/tmp</code> is shared), created and deleted by the runner
What you never touch	<code>~/.claude</code> on the server. Several developers share that Linux user, and the runner uses a temporary config folder on purpose.

3. Install

```
cd backend
npm install --save-exact @anthropic-ai/claude-code@2.1.282
```

Use an exact version, as the CRM does. The CLI's output format and flags change between releases, and the code below depends on them. Upgrade on purpose and re-run the probe in section 11. Do not install it globally: `/usr` is shared by every project on the server.

4. Configure it, and choose the model

backend/.env (server and local)

```
LLM_PROVIDER=claude-cli           # 'mock' (the default) for local dev and tests; see section 7
LLM_BASE_URL=https://api.z.ai/api/anthropic
LLM_API_KEY=<your Z.ai key>        # never commit this
LLM_MODEL=glm-5.3-flash[1m]       # default for every kind of task
LLM_MODEL_WRITE=glm-5.3[1m]       # override ONE kind with the full model
LLM_MODEL_HARD=glm-5.3[1m]        # used only by the escalation helper in section 7
LLM_CONCURRENCY=6                 # calls in flight at once, see section 9
REDIS_DB=7                        # a number nobody else on this Redis uses
```

MODEL ID	USE IT FOR
glm-5.3-flash[1m]	Most work. Classifying, extracting, summarising, short rewrites, simple verdicts, first drafts. Its quota is larger, so it is the cheap default.
glm-5.3[1m]	Hard work. Long structured writing, many rules at once, multi-step research, anything where Flash produced a wrong or invalid answer.

All four names (`glm-5.3-flash` , `glm-5.3-flash[1m]` , `glm-5.3` , `glm-5.3[1m]`) are valid model ids. **With the `[1m]` suffix the context window is 1M tokens. Without it the window is 200K.** Use the `[1m]` names when a prompt can be long. The plain names are enough for short prompts.

Choose the model per task, in one place

```
// llm/models.ts: an explicit map from task kind to the env variable that may override it
const MODEL_ENV = { classify: 'LLM_MODEL_CLASSIFY', write: 'LLM_MODEL_WRITE' } as const;
export type Kind = keyof typeof MODEL_ENV;

export const modelFor = (kind: Kind): string =>
  process.env[MODEL_ENV[kind]] || process.env.LLM_MODEL || 'glm-5.3-flash[1m]';
```

With `LLM_MODEL_WRITE` set, `modelFor('write')` returns the full model and every other kind stays on Flash. Because the map is fixed, a misspelled kind is a compile error and never a silent fallback. Changing the policy is an `.env` edit and a restart, not a code change.

5. The runner

A framework-free module that makes one model call.

It is deliberately small. A production version may also stream events (`--output-format stream-json`) to see which tools ran, stop at the first 401 or 403, send SIGTERM before SIGKILL, and classify every error.

backend/src/llm/claude-runner.ts

```

import { spawn, type ChildProcess } from 'node:child_process';
import { cpSync, mkdirSync, mkdtempSync, readdirSync, readFileSync, rmSync, statSync, writeFileSync } from
'node:fs';
import { tmpdir } from 'node:os';
import { dirname, join } from 'node:path';

export interface RunOptions {
  model: string;          // 'glm-5.3-flash[1m]' (default) or 'glm-5.3[1m]' (hard tasks)
  system: string;         // the rules for this call, written to a file
  prompt: string;         // the input for this call, sent on stdin
  schema?: object;        // JSON Schema (draft-07) for the answer, optional
  skillsDir?: string;     // folder of <name>/SKILL.md to make available for this call
  timeoutMs?: number;     // short jobs: 120_000 is fine. Long writes need about 600_000.
  maxTurns?: number;
}

/** Every child that is running right now, so a shutdown can kill them all. */
const live = new Set<ChildProcess>();
export function killAllChildren(): void {
  for (const c of live) { try { process.kill(-c.pid!, 'SIGKILL'); } catch { /* already gone */ } }
}
process.on('exit', killAllChildren);

/** Scratch folders start with YOUR project's name: /tmp is shared with every other app on the server. */
const SCRATCH_PREFIX = 'myproject-llm-';

/** Call once when the worker starts: removes folders that a crashed worker left behind. */
export function sweepOldScratch(maxAgeMs = 3_600_000): void { // 1 hour: longer than any call's timeout
  for (const name of readdirSync(tmpdir())) {
    if (!name.startsWith(SCRATCH_PREFIX)) continue;
    const dir = join(tmpdir(), name);
    try { if (Date.now() - statSync(dir).mtimeMs > maxAgeMs) rmSync(dir, { recursive: true, force: true }); }
  } catch { /* ignore */ }
}

/** An error that carries the HTTP status, so a caller can decide whether a retry makes sense. */
export class LlmCallError extends Error {
  constructor(message: string, readonly status?: number) { super(message); this.name = 'LlmCallError'; }
}

/** The binary pinned in THIS project's node_modules. Never a global `claude`. */
function claudeBin(): string {
  const pkgJson = require.resolve('@anthropic-ai/claude-code/package.json');
  const pkg = JSON.parse(readFileSync(pkgJson, 'utf8'));
  return join(dirname(pkgJson), typeof pkg.bin === 'string' ? pkg.bin : pkg.bin.claude);
}

export async function runClaude(o: RunOptions): Promise<{ text: string; data?: unknown; raw: any }> {
  const baseUrl = process.env.LLM_BASE_URL;
  const apiKey = process.env.LLM_API_KEY;
  if (!baseUrl || !apiKey) throw new Error('LLM_BASE_URL / LLM_API_KEY are not set');
  const timeoutMs = o.timeoutMs ?? 120_000;

  // A fresh temp folder per call: the working directory, the CLI's config dir, the rules file.
  // (A hard kill can leave it behind: see sweepOldScratch.)
  const root = mkdtempSync(join(tmpdir(), SCRATCH_PREFIX));
  try {
    // cleanup covers the setup steps too
    const cwd = join(root, 'cwd');
    const config = join(root, 'config');
    mkdirSync(cwd);
    mkdirSync(config);
    writeFileSync(join(root, 'system.txt'), o.system);
    if (o.skillsDir) cpSync(o.skillsDir, join(cwd, '.claude', 'skills'), { recursive: true });
  }
}

```

```

const argv = [
  '-p', '--output-format', 'json',
  '--model', o.model,
  '--max-turns', String(o.maxTurns ?? 5),
  '--strict-mcp-config', '--no-session-persistence',
  '--tools', '', // no tools: pure text in, text out
  '--system-prompt-file', join(root, 'system.txt'),
  ...(o.schema ? ['--json-schema', JSON.stringify(o.schema)] : []),
  ...(o.skillsDir ? [] : ['--disable-slash-commands']), // skills need slash commands ON
];

// The child gets ONLY this. Never hand it process.env: that holds your DB URL and secrets.
const env = {
  PATH: process.env.PATH ?? '/usr/bin:/bin',
  ANTHROPIC_BASE_URL: baseUrl, // https://api.z.ai/api/anthropic
  ANTHROPIC_API_KEY: apiKey, // the Z.ai key
  CLAUDE_CONFIG_DIR: config,
  CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: '1',
  API_TIMEOUT_MS: String(timeoutMs),
  CLAUDE_CODE_MAX_RETRIES: '2', // without it a bad key is retried for ~2 minutes
};

const { out, code: exitCode } = await new Promise<{ out: string; code: number | null }>((resolve,
reject) => {
  const child = spawn(claudeBin(), argv, { cwd, env, detached: true, stdio: ['pipe', 'pipe', 'pipe'] });
  live.add(child);
  const killGroup = () => { try { process.kill(-child.pid!, 'SIGKILL'); } catch { /* gone */ } };
  let stdout = '', stderr = '', timedOut = false, tooBig = false;
  const timer = setTimeout(() => { timedOut = true; killGroup(); }, timeoutMs);
  child.stdout!.setEncoding('utf8'); // whole characters, never a split one
  child.stderr!.setEncoding('utf8');
  child.stdout!.on('data', (d: string) => {
    if (tooBig) return;
    stdout += d;
    if (stdout.length > 20e6) { tooBig = true; killGroup(); }
  });
  child.stderr!.on('data', (d: string) => { if (stderr.length < 64_000) stderr += d; });
  child.on('error', (e) => { live.delete(child); clearTimeout(timer); reject(e); });
  child.on('close', (code) => {
    live.delete(child);
    clearTimeout(timer);
    killGroup(); // nothing may outlive the call
    if (timedOut) return reject(new Error(`timeout after ${timeoutMs} ms`));
    if (tooBig) return reject(new Error('CLI output over the 20 MB cap'));
    if (!stdout) return reject(new Error(`CLI exited ${code} with no output`)); // stderr stays out of
errors
    resolve({ out: stdout, code });
  });
  child.stdin!.on('error', () => undefined);
  child.stdin!.end(o.prompt);
});

let raw: any;
try { raw = JSON.parse(out); } // one JSON object (no --verbose!)
catch { throw new Error('CLI output was not JSON'); } // never echo model text into an error
// Errors carry a status and a subtype only: never model text or CLI diagnostics, which can
// hold prompt content and end up persisted by the queue or printed by a logger.
if (raw?.type !== 'result') throw new Error('CLI output was not a result envelope');
if (raw.is_error) {
  const status = typeof raw.api_error_status === 'number' ? raw.api_error_status : undefined;
  throw new LlmCallError(`model error status=${status ?? 'none'} subtype=${raw.subtype ?? 'none'}`,
status);
}
if (exitCode !== 0) throw new Error(`CLI exited ${exitCode} without reporting an error`);
if (o.schema && raw.structured_output == null) {
  throw new Error(`no structured output (${raw.terminal_reason ?? 'unknown'})`);
}

```

```

    }
    return { text: String(raw.result ?? ''), data: raw.structured_output, raw };
  } finally {
    rmSync(root, { recursive: true, force: true }); // the temp folder lives only for this call
  }
}

```

ESM projects: `require.resolve` does not exist there. Add `import { createRequire } from 'node:module'; const require = createRequire(import.meta.url);` at the top.

Shutdown. `killAllChildren()` exists because a PM2 restart or deploy would otherwise leave running `claude` processes behind, still spending quota and holding in-flight slots. Node does not run its `exit` handler when a signal kills the process, and PM2 stops a process with **SIGINT** by default, so the worker must handle both SIGINT and SIGTERM (section 7).

Errors carry no model text. The messages contain only a status and a subtype. Model output and CLI diagnostics can contain prompt content, and queues and loggers persist or print error messages. If you need the diagnostics to debug, log them locally and deliberately.

Timeouts. The default is 120 seconds, enough for a short classification. A long structured write needs about 600 seconds. Pass `timeoutMs` for those, or every long write will fail after spending quota.

Why these flags. `--output-format json` returns one object that includes `structured_output` and `modelUsage`. `--no-session-persistence` and a temp config folder mean the CLI keeps no history. The temp folder does exist on disk while the call runs (it holds the rules file, the skills and the CLI's scratch files) and is deleted afterwards. After a hard kill it can survive, so call `sweepOldScratch()` when the worker starts. It deletes only folders with your project's prefix that are older than an hour. `--tools ''` means the model cannot read files or run commands. `--json-schema` makes the model return data that matches your schema, and you still validate it yourself.

6. Give the model only the tools it needs

Claude Code is built as a coding agent. With no restriction it starts with 19 tools: reading and writing files, editing, searching, running shell commands, fetching web pages, and more. A backend usually needs the model to turn text into text. Every tool you leave on is something the model can use, and so is anything it reads.

The rule: no tools by default.

Start with `--tools ''`. Add one named tool only when the task truly needs it. Never leave Write, Edit, Bash, Grep or Glob available in a backend call. Never bypass permissions: no `--dangerously-skip-permissions` and no bypass mode. Pre-approving exactly the tools you listed with `--allowedTools` is fine.

THE TASK	FLAGS
Text in, text out: classify, extract, write, summarise (almost everything)	<code>--tools ''</code>
Must search or read the web	<code>--tools WebSearch WebFetch --allowedTools WebSearch WebFetch</code> (the CRM's research calls)
Must use the contents of a file	Do not give the model a file tool. Read the file in your own code and put the text in the prompt. A file tool cannot be limited to one folder (see below).

Two flags that look alike. `--tools` decides which tools *exist*. `--allowedTools` only pre-approves tools without asking, and it does not remove the others. Restrict with `--tools .`. Separate tool names with spaces, not commas: a comma list gave no tools at all in this project's probe.

Why it matters

- **The temp folder is not a fence.** With only `Read` enabled and the working directory set to an empty temp folder, the model can still read a file elsewhere on the machine when given its absolute path. File and shell tools run as the same Linux user as your app. On the shared server that user also owns the other projects' files, so assume a model with file or shell tools can reach them.
- **Write, Edit and Bash can change things.** They can alter code, configuration or data, or send data out. A model should never be able to do that in a backend.
- **The model obeys text it reads.** A lead's website, an email or a document can contain instructions written to trick the model. With no tools, the worst result is a wrong answer. Note that an injected instruction can produce a wrong answer that still passes your schema check, so your own application checks must control any consequential action. With Bash it can be a command.
- **Fewer tools also means fewer tokens and fewer mistakes.** The model is not shown tools it cannot use.

Other layers, together

- A fresh temp working folder and config folder per call, so no project files sit beside the model and no history is kept (section 5).
- The environment allowlist, so the database URL and other secrets never exist inside the child (section 5).
- `--strict-mcp-config` with no MCP servers, so no extra tool servers load.
- A separate Linux user for the worker would help, but it is not a fence either: any file that other users can read is still readable. Do not enable file or shell tools in a backend unless the process runs in a real sandbox (a container or a locked-down user with no access to other projects), and test that boundary first.
- The CLI's help (version 2.1.282) lists a `--restricted` mode that removes shell and code-running tools and confines file tools to the working directories. Test it yourself before you rely on it.

A valid answer is not a true or an allowed answer.

A schema check only proves the shape. A model that read an injected instruction can still return a perfectly shaped lie. For any consequential action (sending an email, changing a record, spending money) add your own checks or a human step. Validate the job input too, not only the output.

Check what the model really has

Run a probe with `--output-format stream-json --verbose` and read the first `system` line (subtype `init`). Its `tools` list must be `[]`, or exactly the tools you named. With `--tools ''` the list is empty, with `--tools WebSearch WebFetch` it holds exactly those two, and with no flag it holds 19 tools (Claude Code 2.1.282).

7. Wiring it into Express, Fastify or NestJS

The runner does not care about the framework. What matters is *where* you call it. A call can take minutes, so do not run it inside the HTTP request. The route stores or enqueues a job and answers immediately. A worker process runs the calls.

FRAMEWORK	THE ROUTE	THE WORKER
Express	<code>app.post('/jobs', ...)</code> adds a BullMQ job, returns <code>202</code>	A separate <code>worker.ts</code> with <code>new Worker('llm', handler, { concurrency: 6 })</code>
Fastify	<code>fastify.post('/jobs', ...)</code> does the same	The same <code>worker.ts</code>
NestJS	A controller method calls <code>queue.add</code>	A <code>@Processor</code> class, or a second entry point (<code>worker.ts</code>) that boots Nest without HTTP, as the CRM does

The snippets below need these packages: `npm install bullmq zod dotenv` and `npm install -D ts-node`. Next, choose the real runner or a mock from `LLM_PROVIDER`, defaulting to the mock, so local runs and tests never spend quota. In NestJS this is a provider chosen by `LLM_PROVIDER`.

```
// llm/index.ts
import { runClaude, type RunOptions } from './claude-runner';

async function mockRun(o: RunOptions) {
  // canned answer: no network, no cost
  return { text: 'mock', data: { label: 'mock' }, raw: {} };
}
export const run = process.env.LLM_PROVIDER === 'claude-cli' ? runClaude : mockRun;
```



```

// worker.ts (framework-neutral; BullMQ). Runs as its own process, not inside the web server.
import 'dotenv/config'; // load .env: this process does not get it from the web app
import { UnrecoverableError, Worker } from 'bullmq';
import { z } from 'zod';
import { LlmCallError, killAllChildren, sweepOldScratch } from './llm/claude-runner';
import { run } from './llm';
import { modelFor } from './llm/models';

const rawDb = (process.env.REDIS_DB ?? '').trim(); // Number('') is 0, which would silently pick db 0
if (!/^d+$/.test(rawDb)) throw new Error('REDIS_DB must be set to a whole number');
const redisDb = Number(rawDb);

const jobInput = z.object({ kind: z.enum(['classify', 'write']), text: z.string().max(20_000) });
const answer = z.object({ label: z.string() });
const answerSchema = { type: 'object', properties: { label: { type: 'string' } }, required: ['label'],
additionalProperties: false };

sweepOldScratch(); // remove folders a crashed worker left behind

const worker = new Worker('myproject-llm', async (job) => { // queue name: unique per project
  const { kind, text } = jobInput.parse(job.data); // reject bad input before spending quota
  let r: Awaited<ReturnType<typeof run>>;
  try {
    r = await run({
      model: modelFor(kind),
      system: 'Classify the message. Answer only with the schema.',
      prompt: text,
      schema: answerSchema,
      timeoutMs: kind === 'write' ? 600_000 : 120_000,
    });
  } catch (e) {
    // a wrong key or a forbidden account never heals by retrying
    if (e instanceof LlmCallError && (e.status === 401 || e.status === 403)) throw new
UnrecoverableError(e.message);
    throw e;
  }
  const out = answer.parse(r.data); // validate the shape again in our own code
  // save `out` to your database here
  return out;
}, {
  connection: { host: '127.0.0.1', port: 6379, db: redisDb }, // the same db and queue name in the web app
  concurrency: Number(process.env.LLM_CONCURRENCY ?? 6),
});

// PM2 stops a process with SIGINT by default; a deploy script may send SIGTERM. Handle both, once.
let stopping = false;
async function shutdown() {
  if (stopping) return;
  stopping = true;
  killAllChildren(); // the running model calls fail now, their temp folders are removed
  await worker.close(); // waits for those jobs to settle, no new job is started
  process.exit(0);
}
process.on('SIGINT', shutdown);
process.on('SIGTERM', shutdown);

```

Escalating from Flash to the full model. A suggested pattern. Real errors (a bad key, a rate limit, exhausted quota) must reach you and must not be retried on a more expensive model. Only an answer that fails your own validation is worth one retry:

```

async function runWithEscalation(kind: Kind, o: Omit<RunOptions, 'model'>, validate: (data: unknown) =>
boolean) {
  const first = await run({ ...o, model: modelFor(kind) }); // errors propagate on purpose
  if (validate(first.data)) return first; // Flash was good enough
  const second = await run({ ...o, model: process.env.LLM_MODEL_HARD || 'glm-5.3[1m]' }); // one retry
  if (!validate(second.data)) throw new Error('answer still invalid after escalation');
  return second;
}

```

In NestJS, wrap the runner in an `@Injectable()` service behind an interface (`LlmProvider`) and pick the real or the mock class from `LLM_PROVIDER`.

Shutdown kills the model calls that are running, so those jobs fail on every deploy. Add jobs with a retry and a delay, for example `queue.add('classify', data, { attempts: 2, backoff: { type: 'exponential', delay: 60_000 } })`, so an interrupted job runs again after the restart. The worker above makes a 401 or 403 unrecoverable, because a wrong key never heals. A rate limit (429) gets the delay. If the quota is exhausted (section 9), more retries will fail too: pause the queue by hand until the reset time.

Keep **job status in your database**, not only in Redis. A Redis flush must lose scheduling, never history.

8. Skills

A skill is a folder with a `SKILL.md`: a name, a one-line description, and the instructions. In a normal Claude Code session the CLI finds skills in `~/.claude/skills` and in the project's `.claude/skills`. In the embedded setup there is no `~` to rely on, because the child has a temporary config folder and a temporary working directory. So the app supplies the skills itself.

1. Keep the skill sources in your repo: `backend/llm-skills/email-writer/SKILL.md`. They deploy with the code.
2. For each call, copy that folder into the call's `cwd/.claude/skills` (the runner does this when you pass `skillsDir`). Putting them in `$CLAUDE_CONFIG_DIR/skills` also works.
3. Do **not** pass `--disable-slash-commands` and do not use `--bare`. Both hide skills.
4. Call the skill by name in the prompt: `/email-writer <the input>`. Always start the prompt with your own `/skill-name`. With slash commands on, user text that begins with `/` could trigger a different skill or command.

`backend/llm-skills/email-writer/SKILL.md`

```

---
name: email-writer
description: Use when asked to write a cold email, a follow-up or a reply.
---
Rules:
1. Maximum 90 words, one call to action, no filler opening.
2. Plain, direct tone.
3. Return a subject line, a body, and two alternative subject lines.

```

```

const r = await run({                                     // `run` from llm/index.ts: the real runner or the mock
  model: modelFor('write'),
  system: 'You write sales emails.',
  prompt: '/email-writer to the owner of a roofing company about missed calls',
  schema: emailSchema,                                   // JSON Schema for the CLI
  timeoutMs: 600_000,                                    // a long write, see section 5
  skillsDir: join(__dirname, '../..//llm-skills'),        // the folder that holds every skill
});
const email = emailOut.parse(r.data);                     // emailOut: the zod twin of emailSchema. The mock must
return this shape too.

```

Skill, or a prompt file?

	SYSTEM PROMPT FILE (SYSTEM)	SKILL
Best for	One fixed job per call, text that users edit in the app	Several named jobs the same code can switch between, each with its own rules and examples
Where it lives	Often a database row, written to a temp file for each call	Files in git, deployed with the code
Who can change it	A user, from a screen in the app (for example versioned prompts in a database table)	A developer, by editing the file and deploying

For "a user can update the prompt", keep the text in a database table with versions and pass it as `system`. Skill files change only when someone deploys.

On Flash: invoking by `/name` is reliable and also works with `--tools ''`. Letting the model pick a skill by itself needs the `Skill` tool to be available, which the runner above turns off on purpose (section 6). Built-in skills of the CLI stay listed too. Keep each `SKILL.md` short and concrete, with a fixed output shape.

9. Server rules

- **Limit concurrency.** Z.ai answers HTTP 429 (code 1302) when too many calls are in flight, and does not publish the limit. In our own use, 6 long calls in flight at once ran without a 429 on the Pro plan, so the pool is capped at 6. Your plan may differ. **The limit belongs to the whole Z.ai account**, not to one process: production, staging and anyone's own GLM session on the same key share it.
- **Quota errors are final until the reset.** Z.ai codes 1308 and 1310 mean the 5-hour or weekly credits are used up. Do not retry; pause until the reset time in the message.
- **One worker process, fork mode.** The CRM runs its worker with PM2 `instances: 1`, `exec_mode: 'fork'`. Cluster mode multiplies the concurrency against a single external limit. Example for a new project:

```
// ecosystem.config.cjs (worker only)
module.exports = { apps: [{
  name: 'myproject-worker', script: 'dist/worker.js',
  cwd: '/var/www/html/myproject/backend',
  exec_mode: 'fork', instances: 1,
  kill_timeout: 10000, // time for the SIGINT/SIGTERM handler in section 7 to finish
}] };
```

- **Cap the retries.** `CLAUDE_CODE_MAX_RETRIES` must be set, otherwise a refused key is retried about ten times over two minutes. Treat 401 and 403 as final.
- **Set a timeout and kill the whole process group** (the runner does both), and cap stdout and stderr.
- **Key handling.** The key sits in the server's `.env`, copied there by hand. It never goes in git, in a project `settings.json`, or in a message.
- **Restart only your own processes.** `pm2 restart <your-app>`. Never `pm2 restart all` on the shared server. A changed `.env` needs a restart of the worker, because the worker is the process that spawns the CLI. The app reads `.env` itself, so a plain `pm2 restart` is enough. Do not add `--update-env` to a process whose `PATH` comes from an ecosystem file: it replaces that `PATH` with the shell's.
- **Confirm PM2's stop signal.** PM2 sends SIGINT by default. The worker in section 7 handles SIGINT and SIGTERM, but test a real `pm2 restart` on staging and check afterwards that no `claude` process from the old worker is left.
- **Run `node -v` as the PM2 user** on the server and compare it with what you build with, before the first deploy.
- **Shared `ubuntu` user.** Do not put GLM settings in `~/.claude/settings.json` there. It would silently move every other project that uses Claude Code on the box onto GLM.
- **Log calls in your database:** model, duration, tokens, outcome, error class. Application logs are for stack traces, not for an audit trail.
- **Data protection.** Anything in the prompt goes to Z.ai. Do not send real customer data unless you are allowed to.

10. Traps found in this project

TRAP	WHAT TO DO
<code>--bare</code> removes WebSearch and WebFetch even when named, and it also hides skills	Do not use it for web calls or skill calls
Tool names joined with a comma give you no tools	Space-separate them: <code>--tools WebSearch WebFetch</code>
Without tools the model can invent plausible source URLs	Check every cited URL against the URLs in the real tool results, in code
<code>--json-schema</code> rejects zod's default <code>\$schema</code> (2020-12)	Render the schema as draft-07 and delete the <code>\$schema</code> key
<code>--verbose</code> in a settings file changes the JSON shape	Never read a developer's <code>settings-glm.json</code> at run time. Use env variables.

A <code>[cclaude-code:unrecognized_model]</code> line on stderr	Noise for GLM model names, not an error. Ignore it.
A prompt in argv is readable through <code>ps</code> and argument size is capped	Send the prompt on stdin and the rules as a file
Passing the app's whole environment to the child	Always use the allowlist in section 5
An orphaned <code>cclaude</code> process keeps using your concurrency, especially after a restart	Start it <code>detached</code> , kill the whole group on timeout, and call <code>killAllChildren()</code> on SIGINT, on SIGTERM and on exit
Model text or CLI stderr inside error messages	Queues persist errors and loggers print them. Keep only a status and a subtype.
Sourcing the whole <code>.env</code> into a shell to test	It exports every secret to everything you start. Probe through the runner (section 11).
Leaving the default 19 tools on	Always pass <code>--tools</code> . Start from <code>--tools ''</code> (section 6)
A successful call does not prove which model Z.ai ran	The CLI reports the name you passed. Check speed and quality, and Z.ai's own usage page.

11. Test it

- **Unit and end-to-end tests use a mock provider.** No test calls a real model.
- **One live probe** by hand, to check the key, the model name and the routing. From `backend/`:

`backend/scripts/probe-llm.ts`

```
import { runClaude } from '../src/llm/claude-runner';

runClaude({ model: process.env.LLM_MODEL || 'glm-5.3-flash[1m]', system: 'Be terse.', prompt: 'Reply with the word ok' })
  .then((r) => console.log(r.text, Object.keys(r.raw.modelUsage ?? {})))
  .catch((e) => { console.error(e.message); process.exit(1); });
```

```
cd backend
npx ts-node -r dotenv/config scripts/probe-llm.ts
```

This goes through the same runner, so the same environment allowlist applies: `dotenv` loads `.env` into this one Node process, and the CLI child only receives the allowlisted variables. Do not `source .env` into a shell for a test. That exports every secret to every program you start. Expect `ok [ 'glm-5.3-flash[1m]' ]`. A message with `status=401` means a wrong key. A `model not found` style error means the id changed: read Z.ai's current model list and edit `.env`.

12. Instructions to give an LLM agent

EMBEDDED CLAUDE CODE (GLM) RULES

- The app calls a model by spawning the pinned `claude -p` CLI from backend/node_modules. Never a global claude, never `claude` from PATH, never a library call.
- One call = one child, one fresh temp dir (cwd + CLAUDE_CONFIG_DIR + system.txt), deleted after.
- The child's env is an allowlist: PATH, ANTHROPIC_BASE_URL, ANTHROPIC_API_KEY, CLAUDE_CONFIG_DIR, CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC, API_TIMEOUT_MS, CLAUDE_CODE_MAX_RETRIES. Never process.env.
- Models: default glm-5.3-flash[1m]; use glm-5.3[1m] only for tasks marked hard, chosen by LLM_MODEL / LLM_MODEL_ in .env, not hard-coded.
- TOOLS: always pass --tools. Default is --tools '' (no tools). Add a named tool only when the task needs it. Never Write, Edit, Bash, Grep or Glob in a backend call. Never bypass permissions (no --dangerously-skip-permissions; --allowedTools for exactly the named tools is fine). The temp cwd is NOT a fence: a model with Read can read files by absolute path.
- Prompt on stdin, rules in a file. Validate every answer with a schema in our code.
- Skills: sources in backend/llm-skills/, copied per call to cwd/.claude/skills, invoked as /name. No --bare and no --disable-slash-commands on those calls.
- LLM calls run in the worker, never in an HTTP request. Concurrency capped by LLM_CONCURRENCY. Long writes need timeoutMs ~600000. Handle BOTH SIGINT and SIGTERM (PM2 sends SIGINT): call killAllChildren(), then worker.close(), then exit.
- Validate job input and model output with zod. A valid answer is not a true or allowed one: add checks or a human step before any consequential action.
- Errors carry a status and subtype only, never model text or stderr.
- Never `source .env` into a shell. Probe through the runner (scripts/probe-llm.ts).
- Tests use LLM_PROVIDER=mock. Never put the key in git, in a prompt, in a log or in a message.
- On the shared server: restart only our own pm2 apps, never touch ~/.claude, never pm2 restart all.